

Documentation Technique — FitConnect Administration Java

BTS SIO SLAM — Application Client Lourd

Rédigée pour un étudiant débutant en Java

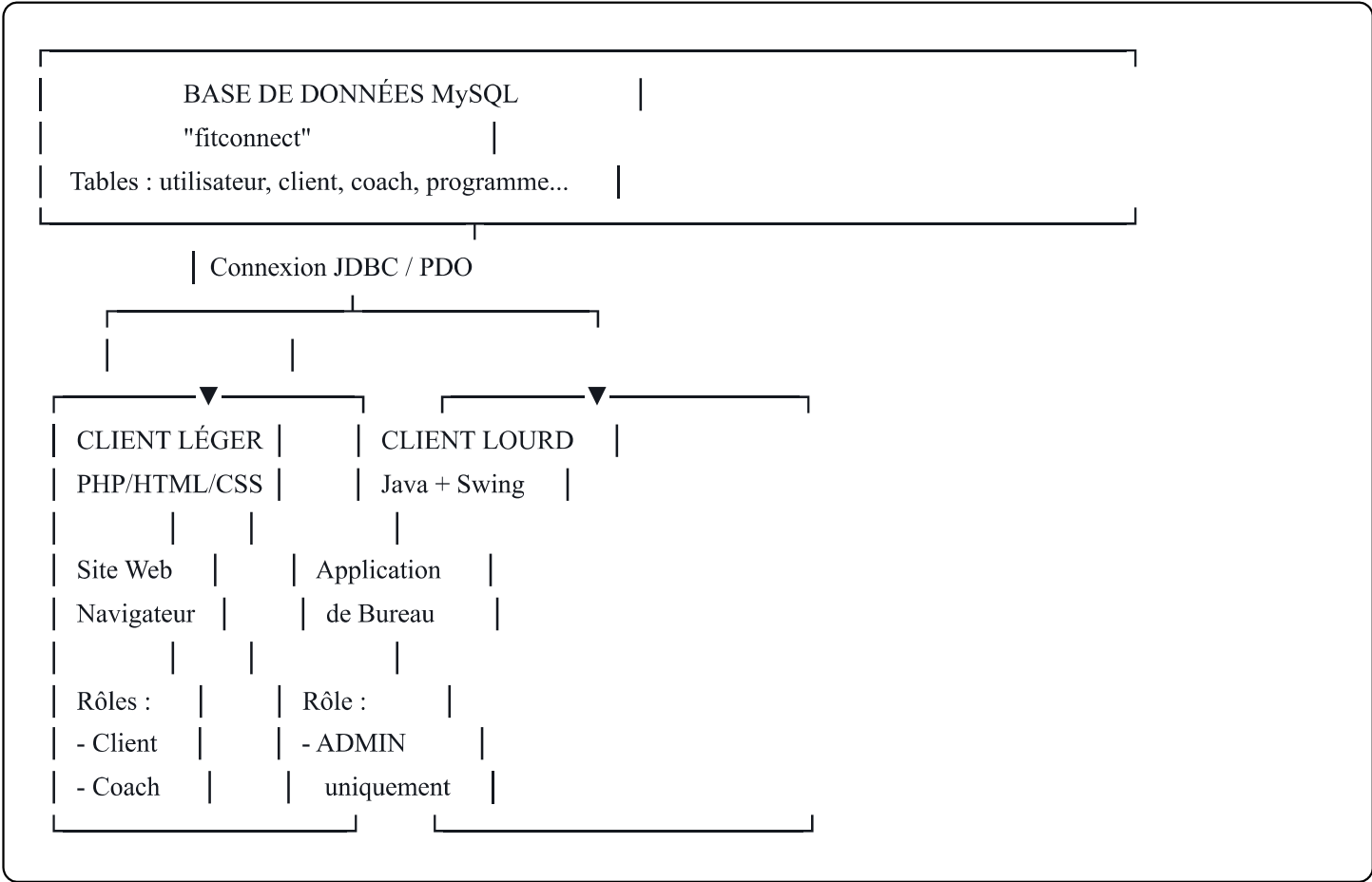
Table des Matières

- 1. Architecture et Écosystème
- 2. Cartographie des Classes et Connexions
- 3. Analyse Technique Vulgarisée
- 4. Guide de Maintenance et Évolutivité
- 5. Glossaire et Dépendances

1. Architecture et Écosystème

1.1 Vue d'ensemble du Projet FitConnect

Le projet FitConnect est composé de **deux applications distinctes** qui partagent la **même base de données MySQL** :



1.2 Rôle de chaque application

Client Léger (PHP) — Le site web :

- Accessible depuis un navigateur (Chrome, Firefox...)
- Permet aux **clients** de s'inscrire, voir leurs programmes
- Permet aux **coachs** de gérer leurs athlètes
- Fonctionne sur un serveur web (XAMPP/MAMP)

Client Lourd (Java) — L'application de bureau :

- S'installe et s'exécute directement sur un ordinateur
- Réservé à l'**administrateur** de la plateforme
- Permet de **valider les coachs** qui s'inscrivent sur le site
- Permet de **gérer les clients** et voir les statistiques
- Fonctionne avec Java + interface graphique Swing

1.3 Comment communiquent-ils ?

Les deux applications **ne se parlent pas directement**. Elles lisent et écrivent toutes les deux dans la **même base de données MySQL**.

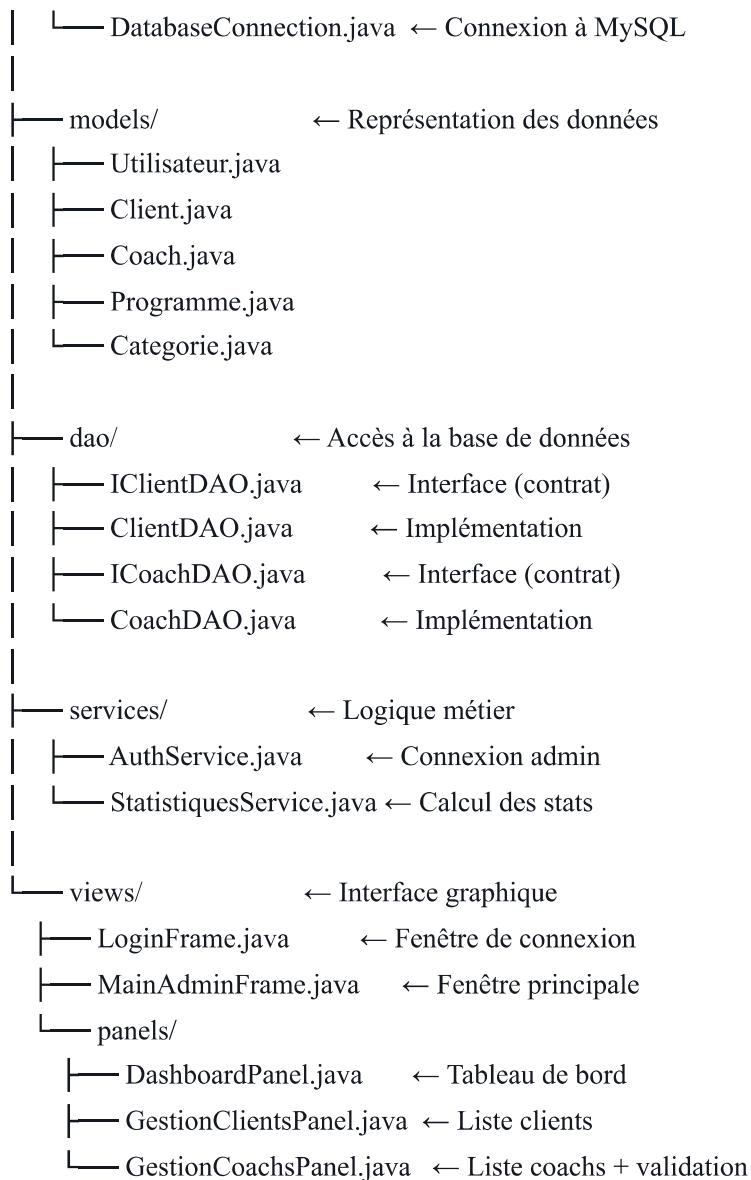
Exemple concret :

1. Un coach s'inscrit sur le SITE WEB (PHP)
→ PHP écrit dans la table "coach" avec valide = 0
2. L'admin ouvre l'APPLICATION JAVA
→ Java lit la table "coach" et voit le coach avec valide = 0
3. L'admin clique "Valider"
→ Java met à jour la table "coach" avec valide = 1
4. Le coach se reconnecte sur le SITE WEB
→ PHP lit valide = 1 → Accès autorisé 

2. Cartographie des Classes et Connexions

2.1 Structure des Packages (Dossiers)

```
src/com/fitconnect/  
|  
├── Main.java      ← Point d'entrée (démarrage)  
|  
└── config/
```



2.2 Description détaillée de chaque classe

Main.java — Le Chef d'Orchestre

Rôle : C'est le point de départ de l'application. Quand tu cliques "Run", Java exécute cette classe en premier.

Ce qu'elle fait :

1. Affiche un message de démarrage dans la console
2. Teste la connexion à la base de données
3. Lance l'interface graphique (LoginFrame)

Interactions :

- Instancie **DatabaseConnection** pour tester la BDD
- Instancie **LoginFrame** pour afficher la fenêtre de connexion

```
// Main.java — Simplifié pour comprendre la logique
```

```
public class Main {  
    public static void main(String[] args) {  
  
        // 1. Tester la connexion BDD  
        DatabaseConnection.getInstance(); // Lance la connexion MySQL  
  
        // 2. Lancer l'interface graphique sur le thread Swing  
        SwingUtilities.invokeLater(() -> {  
            LoginFrame frame = new LoginFrame(); // Crée la fenêtre  
            frame.setVisible(true);           // L'affiche  
        });  
    }  
}
```

`config/DatabaseConnection.java` — Le Portier de la BDD

Rôle : Gère la connexion unique à la base de données MySQL. Utilise le **pattern Singleton** : une seule connexion est créée pour toute l'application.

Analogie : C'est comme une carte d'accès magnétique. On la crée une fois, et tout le monde l'utilise.

Attributs (données stockées) :

- `instance` : L'unique instance de la classe (Singleton)
- `connection` : L'objet représentant la connexion MySQL
- `URL` : L'adresse de la base de données
- `USER` / `PASSWORD` : Identifiants MySQL

Méthodes :

- `getInstance()` : Retourne la connexion existante ou en crée une nouvelle
- `getConnection()` : Donne accès à l'objet Connection JDBC

```
java
```

// DatabaseConnection.java — Expliqué ligne par ligne

```
public class DatabaseConnection {

    // "static" = partagé par toute l'application
    // "private" = inaccessible de l'extérieur
    private static DatabaseConnection instance;
    private Connection connection;

    // L'adresse complète de la base de données
    private static final String URL =
        "jdbc:mysql://localhost:3306/fitconnect?useSSL=false&serverTimezone=UTC";
    // ↑ protocole ↑ serveur ↑ port ↑ nom BDD

    private static final String USER = "root";
    private static final String PASSWORD = "";

    // Constructeur PRIVÉ → On ne peut pas faire "new DatabaseConnection()"
    // depuis l'extérieur. Obligé de passer par getInstance()
    private DatabaseConnection() throws SQLException {
        Class.forName("com.mysql.cj.jdbc.Driver"); // Charge le driver MySQL
        this.connection = DriverManager.getConnection(URL, USER, PASSWORD);
        System.out.println("✓ Connexion à la base de données établie");
    }

    // La seule façon d'obtenir la connexion
    public static DatabaseConnection getInstance() throws SQLException {
        // Si pas encore de connexion OU connexion fermée → en créer une
        if (instance == null || instance.getConnection().isClosed()) {
            instance = new DatabaseConnection(); // Crée UNE SEULE fois
        }
        return instance; // Retourne toujours la même
    }

    public Connection getConnection() {
        return connection; // Donne l'objet Connection aux DAO
    }
}
```

Qui l'utilise : Toutes les classes DAO (ClientDAO, CoachDAO) et AuthService

models/Utilisateur.java — La Carte d'Identité

Rôle : Représente un utilisateur de la BDD en Java. C'est un simple conteneur de données (pas de logique).

Attributs (= colonnes de la table `utilisateur`) :

- `idUtilisateur` → `id_utilisateur`
- `email` → `email`
- `motDePasse` → `mot_de_passe`
- `role` → `role` (CLIENT / COACH / ADMIN)
- `actif` → `actif`
- `dateCreation` → `date_creation`

Méthodes : Uniquement des **getters** (lecture) et **setters** (écriture)

```
java

// Exemple d'utilisation dans un DAO :
Utilisateur user = new Utilisateur(); // Crée un objet vide
user.setEmail("admin@fitconnect.fr"); // Remplit les données
user.setRole("ADMIN");
String email = user.getEmail();      // Lit les données
```

`models/Client.java` — Le Profil Client

Rôle : Représente un client (sportif) avec tous ses attributs physiques et préférences.

Attributs :

- `idClient`, `idUtilisateur`, `nom`, `prenom`
- `poids`, `taille`, `imc` (calculé automatiquement)
- `objectif` (seche / prise_masse / remise_forme / maintien)
- `motivation`, `idCoach`, `idCategorie`

Relation avec Utilisateur : Un Client **contient** un `idUtilisateur` → C'est une **composition** (un client appartient à un utilisateur).

`models/Coach.java` — Le Profil Coach

Rôle : Représente un coach avec ses informations professionnelles.

Attributs importants :

- `valide` : **L'attribut clé** — 0 = en attente, 1 = validé par l'admin
- `dateValidation` : Date à laquelle l'admin a validé
- `specialite`, `anneeExperience`, `cv`

Rôle : Ce sont des **interfaces** Java. Elles définissent **ce que doit faire** un DAO sans dire **comment**.

Analogie : C'est comme un cahier des charges. L'interface dit "tu dois avoir une méthode getAll()", mais ne dit pas comment l'implémenter.

```
java

// ICoachDAO.java — Le contrat
public interface ICoachDAO {
    List<Coach> getAll();    // Récupérer tous les coaches
    List<Coach> getEnAttente(); // Récupérer les coaches non validés
    boolean valider(int id); // Valider un coach
    boolean invalider(int id); // Invalider un coach
    boolean supprimer(int id); // Supprimer un coach
}
```

Pourquoi utiliser une interface ?

- Permet de **changer l'implémentation** sans modifier le reste du code
- Rend le code plus **modulaire** et **testable**
- Respecte les bonnes pratiques de la **POO**

Rôle : Implémentent les interfaces DAO. Contiennent les vraies requêtes SQL pour lire/modifier la base de données.

Relation : `CoachDAO` **implémente** `ICoachDAO` → C'est une **implémentation d'interface**

Méthode clé de `CoachDAO` :

```
java
```

// CoachDAO.java — Méthode valider() expliquée

```
public boolean valider(int idCoach) {  
    // La requête SQL avec paramètre "?" pour éviter les injections  
    String query = "UPDATE coach SET valide = 1, date_validation = NOW() WHERE id_coach = ?";  
  
    try (PreparedStatement pstmt = connection.prepareStatement(query)) {  
        // On remplace le "?" par l'ID du coach  
        pstmt.setInt(1, idCoach);  
  
        // executeUpdate() retourne le nombre de lignes modifiées  
        int lignesModifiees = pstmt.executeUpdate();  
  
        // Si au moins 1 ligne modifiée → succès  
        return lignesModifiees > 0;  
    } catch (SQLException e) {  
        e.printStackTrace();  
        return false; // En cas d'erreur → retourne false  
    }  
}
```

Méthode `getAll()` de ClientDAO :

java


```
// ClientDAO.java — getAll() avec jointures SQL
public List<Client> getAll() {
    List<Client> clients = new ArrayList<>();

    // Requête SQL avec JOIN pour récupérer aussi l'email et la catégorie
    String query = "SELECT c.*, u.email, u.role, cat.nom_categorie " +
        "FROM client c " +
        "INNER JOIN utilisateur u ON c.id_utilisateur = u.id_utilisateur " +
        "LEFT JOIN categorie cat ON c.id_categorie = cat.id_categorie " +
        "ORDER BY c.nom";

    try (Statement stmt = connection.createStatement();
        ResultSet rs = stmt.executeQuery(query)) {

        // Pour chaque ligne retournée par SQL
        while (rs.next()) {
            Client client = new Client(); // Crée un objet Java vide
            client.setIdClient(rs.getInt("id_client")); // Remplit depuis SQL
            client.setNom(rs.getString("nom"));
            client.setPrenom(rs.getString("prenom"));
            client.setEmail(rs.getString("email"));
            client.setPoids(rs.getDouble("poids"));
            clients.add(client); // Ajoute à la liste
        }
    }

    return clients; // Retourne la liste complète
}
```

services/AuthService.java — Le Videur

Rôle : Vérifie si l'email et le mot de passe fournis correspondent à un compte ADMIN dans la BDD.

Interactions :

- Utilise `DatabaseConnection` pour accéder à la BDD
- Utilise `BCrypt` (bibliothèque externe) pour vérifier le mot de passe hashé
- Retourne un objet `Utilisateur` si connexion réussie, ou `null` sinon

java

// AuthService.java — Méthode login() expliquée

```
public Utilisateur login(String email, String password) throws SQLException {

    // Recherche uniquement les admins actifs
    String query = "SELECT * FROM utilisateur WHERE email = ? AND role = 'ADMIN' AND actif = 1";

    try (PreparedStatement pstmt = connection.prepareStatement(query)) {
        pstmt.setString(1, email); // Remplace le "?" par l'email saisi

        try (ResultSet rs = pstmt.executeQuery()) {
            if (rs.next()) { // Si un utilisateur est trouvé
                String hashedPassword = rs.getString("mot_de_passe");

                // BCrypt compare le mot de passe saisi avec le hash en BDD
                // "$2a$" = format compatible Java
                // "$2y$" = format PHP → incompatible directement !
                if (BCrypt.checkpw(password, hashedPassword)) {

                    // Connexion réussie → construire l'objet Utilisateur
                    Utilisateur user = new Utilisateur();
                    user.setIdUtilisateur(rs.getInt("id_utilisateur"));
                    user.setEmail(rs.getString("email"));
                    user.setRole(rs.getString("role"));
                    return user; // Retourne l'utilisateur connecté
                }
            }
        }
    }

    return null; // Retourne null si échec
}
```

services/StatistiquesService.java — Le Comptable

Rôle : Calcule et retourne les statistiques affichées dans le dashboard (nombre de clients, coachs validés, etc.)

Méthodes :

- `getNombreClients()` → COUNT dans la table client
 - `getNombreCoachs()` → COUNT dans la table coach
 - `getNombreCoachsValides()` → COUNT WHERE valide = 1
 - `getNombreCoachsEnAttente()` → COUNT WHERE valide = 0
 - `getNombreProgrammes()` → COUNT dans la table programme
-

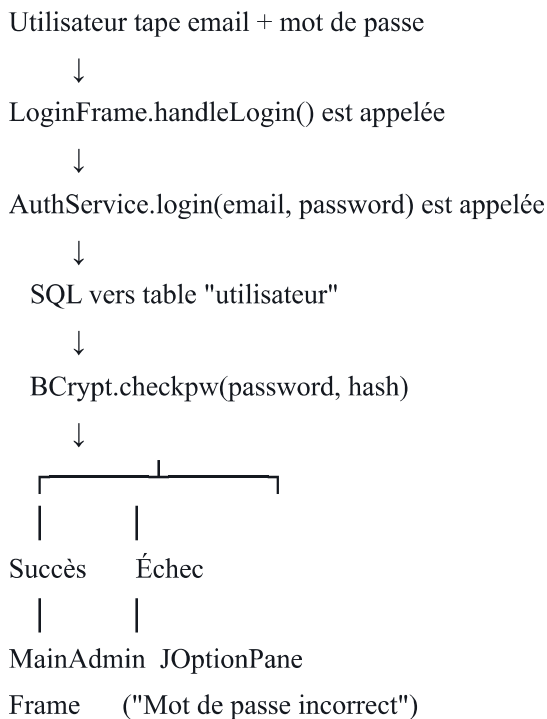
`views/LoginFrame.java` — La Porte d'Entrée

Rôle : La première fenêtre que voit l'utilisateur. Contient les champs email/mot de passe et le bouton "Se connecter".

Interactions :

- Instancie `AuthService` pour vérifier les identifiants
- Si connexion réussie → instancie et affiche `MainAdminFrame`
- Si échec → affiche un message d'erreur

Flux de connexion :



`views/MainAdminFrame.java` — Le Tableau de Bord Principal

Rôle : La fenêtre principale après connexion. Contient un menu latéral et affiche les différents panneaux.

Composants Swing utilisés :

- `JFrame` : La fenêtre principale
- `JPanel` : Les panneaux (sidebar + contenu)
- `CardLayout` : Permet de **switcher entre les panneaux** sans fermer la fenêtre
- `JButton` : Boutons du menu latéral

Interactions :

- Instancie `DashboardPanel`, `GestionClientsPanel`, `GestionCoachsPanel`

- Gère la navigation entre les panneaux via `CardLayout`

`views/panels/DashboardPanel.java` — L'Accueil Admin

Rôle : Affiche 5 cartes de statistiques (Clients total, Coachs total, Coachs validés, En attente, Programmes).

Interactions :

- Instancie `StatistiquesService` pour récupérer les chiffres
- Crée des `JLabel` et `JPanel` colorés pour afficher les stats

`views/panels/GestionCoachsPanel.java` — Le Panneau de Validation

Rôle : Panneau le plus important pour l'admin. Affiche la liste des coachs et permet de les valider/invalidier.

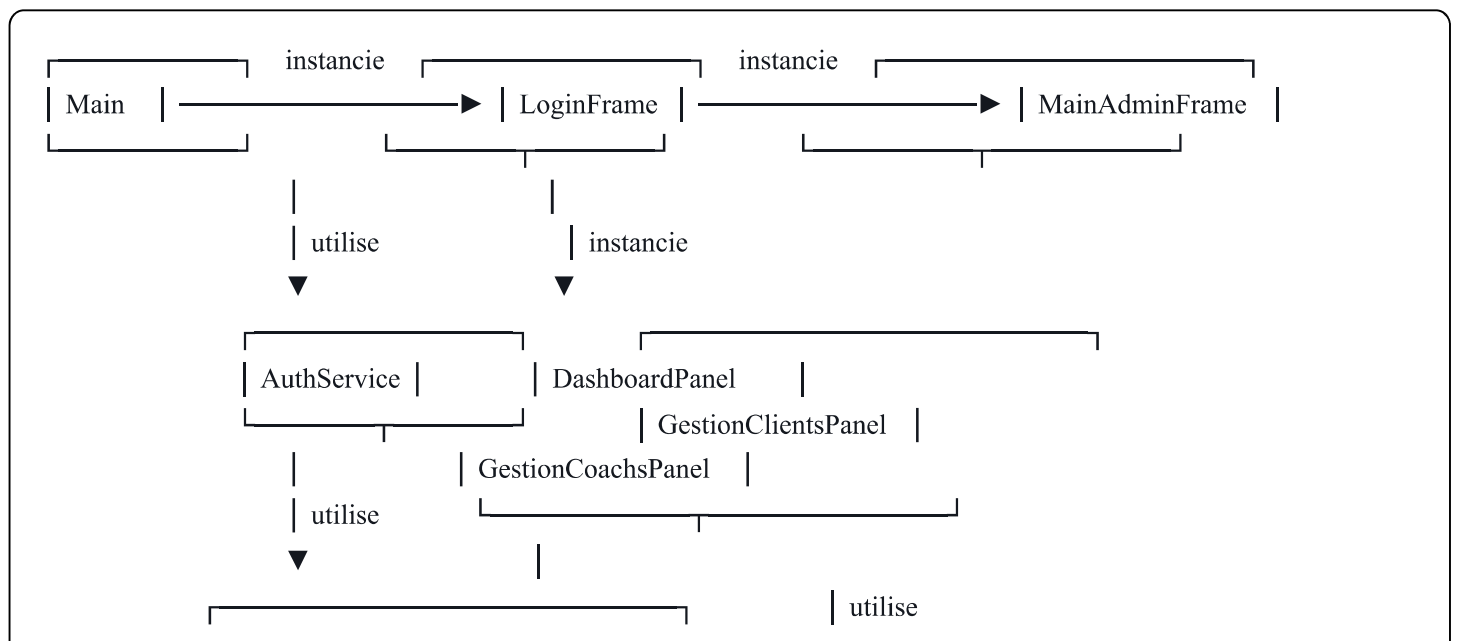
Composants Swing :

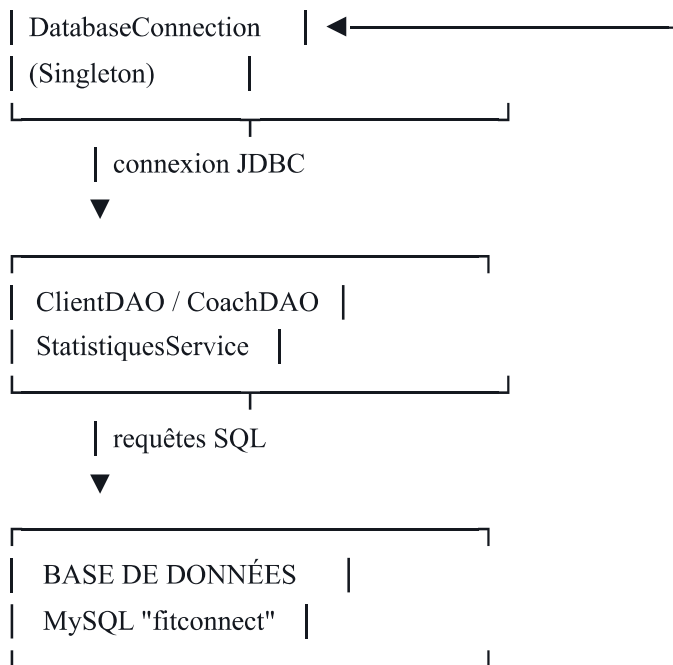
- `JTable` : Tableau affichant la liste des coachs
- `JComboBox` : Filtre (Tous / Validés / En attente)
- Boutons : Valider, Invalidier, Supprimer

Interactions :

- Instancie `CoachDAO` pour lire les coachs depuis la BDD
- Appelle `CoachDAO.valider(id)` quand l'admin clique "Valider"
- Appelle `CoachDAO.supprimer(id)` quand l'admin clique "Supprimer"

2.3 Diagramme des Interactions





2.4 Flux Complet : "Valider un Coach"

Voici exactement ce qui se passe quand l'admin clique sur "Valider" :

ÉTAPE 1 — L'admin clique sur "Valider" dans GestionCoachsPanel

→ ActionListener du bouton est déclenché

ÉTAPE 2 — GestionCoachsPanel récupère l'ID du coach sélectionné

→ `int idCoach = (int) tableModel.getValueAt(row, 0);`

ÉTAPE 3 — GestionCoachsPanel appelle CoachDAO

→ `boolean succes = coachDAO.valider(idCoach);`

ÉTAPE 4 — CoachDAO construit la requête SQL

→ `"UPDATE coach SET valide=1, date_validation=NOW() WHERE id_coach=?"`

ÉTAPE 5 — CoachDAO utilise DatabaseConnection pour envoyer la requête

→ `connection.prepareStatement(query)`

→ `pstmt.executeUpdate()`

ÉTAPE 6 — MySQL met à jour la table coach

→ La ligne du coach est modifiée : valide = 1

ÉTAPE 7 — CoachDAO retourne true à GestionCoachsPanel

ÉTAPE 8 — GestionCoachsPanel rafraîchit la JTable

→ `loadData()` est appelée

→ La liste se met à jour dans l'interface

ÉTAPE 9 — Sur le site PHP, le coach peut maintenant se connecter

→ PHP lit valide=1 → Accès autorisé 

3. Analyse Technique Vulgarisée

3.1 JDBC — Comment Java parle à MySQL

JDBC (Java Database Connectivity) est l'API Java qui permet de se connecter à une base de données. C'est comme un interprète entre Java et MySQL.

Les 4 objets JDBC essentiels :

Objet	Rôle	Analogie
Connection	La connexion ouverte à MySQL	Le fil téléphonique
PreparedStatement	Une requête SQL préparée	La lettre à envoyer
ResultSet	Les résultats de la requête	La réponse reçue
DriverManager	Gère les connexions disponibles	L'opérateur téléphonique

```

java

// Exemple complet d'une requête JDBC commentée
Connection conn = DatabaseConnection.getInstance().getConnection();
//      ↑ objet connexion à MySQL

String sql = "SELECT * FROM coach WHERE valide = ?";
//                        ↑ paramètre sécurisé

PreparedStatement pstmt = conn.prepareStatement(sql);
//      ↑ prépare la requête (évite les injections SQL)

pstmt.setInt(1, 0); // Le premier "?" vaut 0 (non validé)

ResultSet rs = pstmt.executeQuery();
//      ↑ envoie la requête, reçoit les résultats

while (rs.next()) { // Pour chaque ligne retournée
    String nom = rs.getString("nom"); // Lit la colonne "nom"
    int id = rs.getInt("id_coach"); // Lit la colonne "id_coach"
}

```

3.2 POO — Les 4 Piliers Appliqués

Encapsulation : Les attributs des modèles sont `private` et accessibles uniquement via des getters/setters.

```

java

// Dans Coach.java
private int idCoach;    // Inaccessible directement
private String nom;     // Inaccessible directement
private boolean valide; // Inaccessible directement

public int getIdCoach() { return idCoach; } // Accès en lecture
public void setNom(String n) { this.nom = n; } // Accès en écriture

```

Abstraction (via les Interfaces) : Les interfaces `IClientDAO` et `ICoachDAO` définissent **ce que font** les DAO sans dire **comment**.

Héritage : `LoginFrame` et `MainAdminFrame` héritent de `JFrame` (classe Swing) → Elles **sont** des fenêtres avec toutes les propriétés d'une fenêtre.

```
java

public class LoginFrame extends JFrame { // LoginFrame EST une JFrame
    // Elle hérite de : setTitle(), setSize(), setVisible(), etc.
}
```

Polymorphisme : `CoachDAO` implémente `ICoachDAO` → On peut utiliser `ICoachDAO dao = new CoachDAO()`. Si demain on veut changer la base de données, on crée `CoachDAOPostgres implements ICoachDAO` sans toucher au reste.

3.3 Pattern Singleton — Une Seule Connexion

Le Singleton garantit qu'il n'existe **qu'une seule instance** de `DatabaseConnection`. Cela évite d'ouvrir 50 connexions MySQL simultanément.

```
java

// Sans Singleton (MAUVAIS) :
DatabaseConnection conn1 = new DatabaseConnection(); // Connexion 1
DatabaseConnection conn2 = new DatabaseConnection(); // Connexion 2
DatabaseConnection conn3 = new DatabaseConnection(); // Connexion 3
// → 3 connexions ouvertes, gaspillage de ressources

// Avec Singleton (BON) :
DatabaseConnection conn1 = DatabaseConnection.getInstance(); // Connexion 1
DatabaseConnection conn2 = DatabaseConnection.getInstance(); // Même connexion 1
DatabaseConnection conn3 = DatabaseConnection.getInstance(); // Même connexion 1
// → 1 seule connexion, efficace
```

3.4 Pattern DAO — Séparation des Responsabilités

Le **DAO** (Data Access Object) isole le code SQL du reste de l'application. Si demain on veut passer de MySQL à PostgreSQL, on modifie **uniquement les DAO**, pas les vues ni les services.

```
SANS DAO (MAUVAIS) :
GestionCoachsPanel.java contient du SQL → couplage fort, difficile à maintenir

AVEC DAO (BON) :
GestionCoachsPanel → appelle CoachDAO → contient le SQL
Si on change la BDD → on modifie seulement CoachDAO ✓
```


3.5 Swing — L'Interface Graphique

Swing est la bibliothèque Java pour créer des interfaces graphiques de bureau (fenêtres, boutons, tableaux...).

Composants utilisés dans FitConnect :

Composant	Utilisation dans l'app
JFrame	Fenêtre principale (LoginFrame, MainAdminFrame)
JPanel	Conteneurs / sections de la fenêtre
JLabel	Textes affichés (titres, valeurs stats)
TextField	Champ de saisie (email)
PasswordField	Champ mot de passe (masqué)
Button	Boutons (Se connecter, Valider, Supprimer)
JTable	Tableaux (liste clients, liste coaches)
JComboBox	Liste déroulante (filtre Tous/Validés/En attente)
CardLayout	Navigation entre panneaux sans rechargement

3.6 BCrypt — Le Gardien des Mots de Passe

BCrypt est un algorithme de **hachage** des mots de passe. Un hash est une transformation **irréversible** : on ne peut pas retrouver le mot de passe original depuis le hash.

```
java

// Hacher un mot de passe (lors de la création)
String motDePasse = "test";
String hash = BCrypt.hashpw(motDePasse, BCrypt.gensalt(10));
// hash = "$2a$10$B5PyXlmBGkBKq4TQ4xixOu..." (60 caractères)

// Vérifier un mot de passe (lors de la connexion)
boolean correct = BCrypt.checkpw("test", hash);
// → true si le mot de passe correspond au hash
```

Pourquoi 2y (PHP) ≠ 2a (Java) ? PHP génère des hash avec le préfixe `$2y$`. Java n'accepte que `$2a$`. C'est pour cette raison que la base de données doit contenir des hash `$2a$` pour que la connexion Java fonctionne.

4. Guide de Maintenance et Évolutivité

4.1 Modification 1 — Ajouter une Colonne "Téléphone" dans la Liste des Clients

Contexte : Tu veux afficher le numéro de téléphone des clients dans le tableau de `GestionClientsPanel`.

Fichier à modifier : `src/com/fitconnect/views/panels/GestionClientsPanel.java`

Étape 1 — Modifier les colonnes du tableau :

```
java

// Trouver la ligne qui définit les colonnes (chercher "columnNames")
String[] columnNames = {"ID", "Nom", "Prénom", "Email", "Objectif", "Coach", "Catégorie"};
// Ajouter "Téléphone" :
String[] columnNames = {"ID", "Nom", "Prénom", "Email", "Objectif", "Coach", "Catégorie", "Téléphone"};
```

Étape 2 — Modifier la requête SQL dans ClientDAO.java :

```
java

// Dans ClientDAO.java, méthode getAll()
// Ajouter c.telephone dans le SELECT :
String query = "SELECT c.id_client, c.nom, c.prenom, u.email, " +
               "c.objectif, co.nom AS nom_coach, cat.nom_categorie, " +
               "c.telephone " + // ← Ajouter cette ligne
               "FROM client c ...";
```

Étape 3 — Remplir la nouvelle colonne dans GestionClientsPanel.java :

```
java

// Dans la boucle qui remplit le tableau, ajouter :
row[7] = client.getTelephone(); // Index 7 = 8ème colonne
```

Étape 4 — Ajouter getter/setter dans Client.java :

```
java

private String telephone;
public String getTelephone() { return telephone; }
public void setTelephone(String t) { this.telephone = t; }
```

4.2 Modification 2 — Changer la Règle de Calcul de l'IMC

Contexte : L'IMC est actuellement calculé en BDD (colonne GENERATED). Tu veux le calculer en Java et afficher une catégorie ("Normal", "Surpoids"...).

Fichier à modifier : `src/com/fitconnect/models/Client.java`

Ajouter une méthode de calcul :

```
java

// Dans Client.java, ajouter cette méthode :
public String getCategorieIMC() {
    if (imc < 18.5) return "Insuffisance pondérale";
    if (imc < 25.0) return "Poids normal";
    if (imc < 30.0) return "Surpoids";
    return "Obésité";
}

// Utilisation dans GestionClientsPanel.java :
String categorie = client.getCategorieIMC(); // "Poids normal"
```

4.3 Modification 3 — Ajouter un Bouton "Envoyer un Email" à un Coach

Contexte : Tu veux ajouter un bouton dans `GestionCoachsPanel` pour envoyer un email au coach sélectionné.

Fichier à modifier : `src/com/fitconnect/views/panels/GestionCoachsPanel.java`

Étape 1 — Ajouter le bouton dans l'interface :

```
java

// Trouver la zone où sont déclarés les autres boutons
JButton btnValider = new JButton("✅ Valider");
JButton btnInvalidier = new JButton("❌ Invalidier");
JButton btnSupprimer = new JButton("🗑 Supprimer");

// Ajouter le nouveau bouton :
JButton btnEmail = new JButton("✉ Envoyer Email");
btnPanel.add(btnEmail); // L'ajouter au panneau de boutons
```

Étape 2 — Ajouter l'action du bouton :

```
java
```

```
btnEmail.addActionListener(e -> {
    int row = table.getSelectedRow(); // Récupère la ligne sélectionnée
    if (row >= 0) {
        String email = (String) tableModel.getValueAt(row, 2); // Colonne email
        // Ouvre le client mail par défaut
        try {
            Desktop.getDesktop().mail(new URI("mailto:" + email));
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Impossible d'ouvrir le client mail");
        }
    }
});
```

4.4 Modification 4 — Changer les Identifiants de Connexion BDD

Contexte : Tu veux changer le mot de passe MySQL (ex: pour un serveur de production).

Fichier à modifier : `src/com/fitconnect/config/DatabaseConnection.java`

```
java

// Lignes à modifier :
private static final String URL = "jdbc:mysql://localhost:3306/fitconnect?useSSL=false&serverTimezone=UTC";
//
//      ↑ Changer si serveur distant
private static final String USER = "root"; // ← Changer le nom d'utilisateur
private static final String PASSWORD = ""; // ← Changer le mot de passe
```

5. Glossaire et Dépendances

5.1 Bibliothèques Externes

`mysql-connector-j-8.0.33.jar`

Rôle : Permet à Java de se connecter à MySQL. Sans cette bibliothèque, Java ne sait pas parler à une base de données MySQL.

Fichier : `lib/mysql-connector-j-8.0.33.jar`

Utilisation dans le code :

```
java

Class.forName("com.mysql.cj.jdbc.Driver"); // Charge le driver
DriverManager.getConnection(URL, USER, PASSWORD); // Ouvre la connexion
```

Où télécharger : <https://repo1.maven.org/maven2/com/mysql/mysql-connector-j/8.0.33/mysql-connector-j-8.0.33.jar>

bcrypt-0.4.jar

Rôle : Permet de hasher et vérifier des mots de passe avec l'algorithme BCrypt. Indispensable pour la sécurité (on ne stocke jamais un mot de passe en clair).

Fichier : `lib/jbcrypt-0.4.jar`

Utilisation dans le code :

```
java  
  
BCrypt.hashpw(password, BCrypt.gensalt(10)); // Hasher  
BCrypt.checkpw(password, hash);           // Vérifier
```

Où télécharger : <https://repo1.maven.org/maven2/org/mindrot/jbcrypt/0.4/jbcrypt-0.4.jar>

5.2 Glossaire BTS SIO

Terme	Définition simple
Persistence	Le fait que les données survivent à la fermeture du programme (grâce à la BDD)
Objet	Instance d'une classe. Ex : <code>new Client()</code> crée un objet Client
Attribut	Variable appartenant à une classe. Ex : <code>nom</code> dans <code>Client.java</code>
Méthode	Fonction appartenant à une classe. Ex : <code>getAll()</code> dans <code>ClientDAO</code>
Classe	Le moule qui définit la structure d'un objet
Instance	Un objet créé à partir d'une classe
Interface	Contrat définissant les méthodes à implémenter
Héritage	Une classe reprend les attributs/méthodes d'une autre (<code>extends</code>)
Implémentation	Une classe réalise le contrat d'une interface (<code>implements</code>)
Composition	Une classe contient une instance d'une autre classe
Encapsulation	Protéger les données avec <code>private</code> + getters/setters
DAO	Data Access Object — classe dédiée aux requêtes SQL
MVC	Model-View-Controller — séparation données / affichage / logique
Singleton	Pattern garantissant une seule instance d'une classe

Terme	Définition simple
JDBC	API Java pour communiquer avec une base de données
PreparedStatement	Requête SQL préparée, protégée contre les injections SQL
ResultSet	Objet contenant les résultats d'une requête SQL
BCrypt	Algorithme de hachage sécurisé pour les mots de passe
Hash	Transformation irréversible d'une chaîne (mot de passe → code)
CardLayout	Gestionnaire de mise en page Swing pour switcher entre panneaux
JTable	Composant Swing affichant des données en tableau
Thread Swing (EDT)	Fil d'exécution dédié à l'interface graphique Java

5.3 Récapitulatif des Fichiers par Rôle

Fichier	Package	Rôle Principal
Main.java	com.fitconnect	Point d'entrée — démarre l'app
DatabaseConnection.java	config	Connexion MySQL (Singleton)
Utilisateur.java	models	Modèle table utilisateur
Client.java	models	Modèle table client
Coach.java	models	Modèle table coach
Programme.java	models	Modèle table programme
Categorie.java	models	Modèle table categorie
IClientDAO.java	dao	Interface contrat Client
ClientDAO.java	dao	Requêtes SQL table client
ICoachDAO.java	dao	Interface contrat Coach
CoachDAO.java	dao	Requêtes SQL table coach
AuthService.java	services	Authentification BCrypt
StatistiquesService.java	services	Calcul des statistiques
LoginFrame.java	views	Fenêtre de connexion

Fichier	Package	Rôle Principal
MainAdminFrame.java	views	Fenêtre principale admin
DashboardPanel.java	views/panels	Tableau de bord stats
GestionClientsPanel.java	views/panels	Gestion des clients
GestionCoachsPanel.java	views/panels	Validation des coachs

Documentation rédigée pour le BTS SIO SLAM — Projet FitConnect
Application d'Administration Java — Client Lourd